COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	10/07/2026	**DATE SUBMITTED:**	10/07/2026
NAME:	Irish Crystle C. Prieto Jullian Edrain Sevilla	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

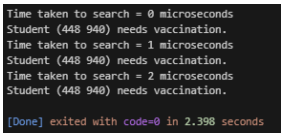
Copy and paste the questions from the lab and include your answers after.

ILO A

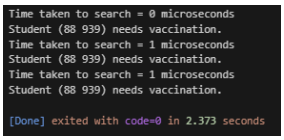
Analysis:
- The algorithm has a quadratic worst-case time complexity, $O(n^2)$.
- The graph shows a curve that rises increasingly steeply, not a straight line — this is characteristic of n^2 growth.
- As the input size n doubles, the number of operations roughly quadruples:
  - n = 10 ⟶ 110 operations
  - n = 20 ⟶ 420 operations (≈ 4× increase)
  - n = 30 ⟶ 930 operations (≈ 9× from n=10)
- The formula $n^2 + n$ means the n^2 term dominates as n grows larger; the +n term becomes negligible for large inputs.
- This is much less efficient than linear O(n) or logarithmic O(log n) algorithms, and will slow down significantly for large arrays.

ILO B

Prieto

Input size N	Execution speed	Screenshot	Observations
1,000	0 microseconds		Search is instant,10 steps for N=1,000.
10,000	1 microseconds		Only 1 sec increase despite a 10 times larger list of 14 steps.
100,000	2 microseconds		Still extremely fast; ~17 steps for 100times bigger list.

Sevilla

Input size N	Execution speed	Screenshot	Observations
1,000	0 microseconds		Search is instantaneous; binary search completes in 10 steps.
10,000	1 microseconds		Slight increase,still extremely fast, only 14 steps despite 10times larger input.
100,000	1microseconds		No further increase in measured time,17 steps for 100times bigger input.

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime
performance by comparing experimental and theoretical analysis

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Did not replace the original thinking**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

ILOB:

Welcome | SevillaMain.cpp

Act11 > SevillaMain.cpp > needs_vaccination(Student, vector<Student>&)

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
#include <cmath>

using namespace std;

class Student {
private:
    pair<int, int> name;
    bool vaccinated;

public:
    Student(pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    pair<int, int> get_name() const { return name; }
    bool is_vaccinated() const { return vaccinated; }

    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};

auto generate_random_Student(int max) {
    random_device rd;
    mt19937 rand(rd());
    uniform_int_distribution<mt19937::result_type> uniform_dist(1, max);

    auto random_name = make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

bool needs_vaccination(Student p, vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (true) {
        auto range_length = distance(first, last);
        auto mid_element_index = floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        if (mid_element == p && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == p && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > p)
            advance(last, -mid_element_index);
```

```
59              advance(last, -mid_element_index);
60          else if (mid_element < p)
61              advance(first, mid_element_index);
62
63          if (range_length == 1)
64              return true;
65      }
66  }
67
68  void search_test(int size, Student p) {
69      vector<Student> people;
70
71      for (auto i = 0; i < size; i++)
72          people.push_back(generate_random_Student(size));
73
74      sort(people.begin(), people.end());
75
76      auto begin = chrono::steady_clock::now();
77
78      bool search_result = needs_vaccination(p, people);
79
80      auto end = chrono::steady_clock::now();
81      auto elapsed = chrono::duration_cast<chrono::microseconds>(end - begin).count();
82
83      cout << "Time taken to search = " << elapsed << " microseconds" << endl;
84      cout << "Student (" << p.get_name().first << " " << p.get_name().second << ") ";
85      if (search_result)
86          cout << "needs vaccination." << endl;
87      else
88          cout << "does not need vaccination." << endl;
89  }
90
91  int main() {
92      auto p = generate_random_Student(1000);
93      search_test(1000, p);
94      search_test(10000, p);
95      search_test(100000, p);
96      return 0;
97  }
```

Welcome | Prietomain.cpp

Prietomain.cpp > Student

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
#include <cmath>


using namespace std;


class Student {
private:
    pair<int, int> name;
    bool vaccinated;


public:
    Student(pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    pair<int, int> get_name() const { return name; }
    bool is_vaccinated() const { return vaccinated; }


    bool operator==(const Student& p) const {
        return this->name == p.name;
    }


    bool operator<(const Student& p) const {
        return this->name < p.name;
    }


    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};


auto generate_random_Student(int max) {
    random_device rd;
    mt19937 rand(rd());
    uniform_int_distribution<mt19937::result_type> uniform_dist(1, max);


    auto random_name = make_pair(uniform_dist(rand), uniform_dist(rand));
```

```
48      auto random_name = make_pair(uniform_dist(rand), uniform_dist(rand));
49      bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
50      return Student(random_name, is_vaccinated);
51  }
52
53
54  bool needs_vaccination(Student p, vector<Student>& people) {
55      auto first = people.begin();
56      auto last = people.end();
57
58
59      while (true) {
60          auto range_length = distance(first, last);
61          auto mid_element_index = floor(range_length / 2);
62          auto mid_element = *(first + mid_element_index);
63
64
65          if (mid_element == p && mid_element.is_vaccinated() == false)
66              return true;
67          else if (mid_element == p && mid_element.is_vaccinated() == true)
68              return false;
69          else if (mid_element > p)
70              advance(last, -mid_element_index);
71          else if (mid_element < p)
72              advance(first, mid_element_index);
73
74
75          if (range_length == 1)
76              return true;
77      }
78  }
79
80
81  void search_test(int size, Student p) {
82      vector<Student> people;
83
84
85      for (auto i = 0; i < size; i++)
86          people.push_back(generate_random_Student(size));
87
88
89      sort(people.begin(), people.end());
90
91
92      auto begin = chrono::steady_clock::now();
```

```cpp
    auto begin = chrono::steady_clock::now();


    bool search_result = needs_vaccination(p, people);


    auto end = chrono::steady_clock::now();
    auto elapsed = chrono::duration_cast<chrono::microseconds>(end - begin).count();


    cout << "Time taken to search = " << elapsed << " microseconds" << endl;
    cout << "Student (" << p.get_name().first << " " << p.get_name().second << ") ";
    if (search_result)
        cout << "needs vaccination." << endl;
    else
        cout << "does not need vaccination." << endl;
}


int main() {
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Supplementary: Sevilla

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Theoretical Analysis	The algorithm uses two nested loops to compare every pair of elements: Outer loop (i) runs n times Inner loop (j) runs from i+1 to n-1, so it checks fewer elements each time In the worst case (all elements are unique)

Input Size	Comparisons (Worst case)
10	45
20	190
50	1,255
100	4,950

Observation: When the input size doubles, the number of comparisons increases by about 4 times. This is the signature of quadratic growth.
Best case: Duplicate found at the first pair = O(1)
Worst case: All elements are unique = O(n²)

Experimental Analysis:

CPE010_HOA3p1_SevillaPrieto_sc4.cpp

```cpp
#include <iostream>
#include <chrono>
using namespace std;

bool Unique(int A[], int n) {
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (A[i] == A[j])
                return false; // duplicate found
    return true; // all unique
}

int main() {
    cout << "n\tTime (microseconds)\tResult\n";

    int sizes[] = {100, 500, 1000, 2000};
    for (int n : sizes) {
        int* A = new int[n];
        for (int i = 0; i < n; i++) A[i] = i;

        auto start = chrono::high_resolution_clock::now();
        bool result = Unique(A, n);
        auto end = chrono::high_resolution_clock::now();

        auto time = chrono::duration_cast<chrono::microseconds>(end - start).count();
        cout << n << "\t" << time << "\t\t\t" << (result ? "Unique" : "Duplicate") << "\n";

        delete[] A;
    }
    return 0;
}
```

COMPUTER ENGINEERING

QUEZON CITY

```
[Running] cd "c:\Users\Jullian Sevilla\One
"c:\Users\Jullian Sevilla\OneDrive\Document
n   Time (microseconds) Result
100 11          Unique
500 266         Unique
1000    1028            Unique
2000    3842            Unique

[Done] exited with code=0 in 2.033 seconds
```

Input Size	Time	Result
100	11	Unique
500	266	Unique
1,000	1028	Unique
2,000	3842	Unique

Analysis and Comparison:

Aspect	Finding
Theoretical Complexity	$O(n^2)$ — Quadratic
Experimental Trend	Time grows roughly 4× when n doubles
Match	Experimental results agree with $O(n^2)$ theoretical prediction
Strengths	Simple to understand, no extra memory needed
Weaknesses	Becomes very slow when n is large; not practical for big datasets
Better Alternative	Sort the array first, then check only adjacent elements = O(n log n)

Conclusion:

The given algorithm correctly checks for uniqueness but is inefficient for large inputs. While it works fine for small arrays, its quadratic complexity means it will take too long for thousands or millions of elements. Using a sorting step before comparison gives a much better O(n log n) performance and should be preferred in real applications.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.


```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Part A:
Theoretical Analysis:
Each call reduces n by exactly 1
To compute x^n, we need n recursive calls

Exponent	Recursive Calls	Observation
5	5	N step
10	10	N step
20	20	N step
100	100	N step
1,000	1,000	N step

CPE010_HOA3p1_SevillaPrieto_sc4.cpp | CPE010_HOA3p1_SevillaPrieto_sc5.cpp | CPE010_HOA3

CPE010_HOA3p1_SevillaPrieto_sc5.cpp > main()

```cpp
#include <iostream>
using namespace std;

long long rpower(int x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

int main() {
    cout << "rpower: Linear O(n)\n";
    cout << "x\tn\tResult\t\tSteps\n";
    int x = 2;
    for (int n : {5, 10, 20, 50, 100}) {
        cout << x << "\t" << n << "\t" << rpower(x, n) << "\t\t" << n << "\n";
    }
    return 0;
}
```

```
[Running] cd "c:\Users\Jullian Sevilla\OneD
"c:\Users\Jullian Sevilla\OneDrive\Document
rpower: Linear O(n)
x   n   Result      Steps
2   5   32      5
2   10  1024        10
2   20  1048576     20
2   50  1125899906842624       50
2   100 0       100

[Done] exited with code=0 in 1.038 seconds
```

Part B:
Theoretical Analysis:
Each call halves the value of n
Number of steps = log2(n)
Complexity: O(log n) = Logarithmic

Exponent	Step log2(n)	Observation
5	3	Very few steps
10	4	+1 step only
20	5	+1 step only

100	7	Still very small
1,000	10	Almost no growth
1,000,000	20	Only 20 steps

Experimental Analysis:

CPE010_HOA3p1_SevillaPrieto_sc4.cpp | CPE010_HOA3p1_SevillaPrieto_sc5.cpp

CPE010_HOA3p1_SevillaPrieto_sc5.cpp > brpower(int, int)

```cpp
#include <iostream>
using namespace std;

long long brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 == 1) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        long long y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    cout << "=== brpower: Logarithmic O(log n) ===\n";
    cout << "x\tn\tResult\t\tSteps ≈ log2(n)\n";
    int x = 2;
    for (int n : {5, 10, 20, 100, 1000}) {
        cout << x << "\t" << n << "\t" << brpower(x, n) << "\t\t"
             << (31 - __builtin_clz(n)) << "\n";
    }
    return 0;
}
```

```
[Running] cd "c:\Users\Jullian Sevilla\OneD
"c:\Users\Jullian Sevilla\OneDrive\Document
=== brpower: Logarithmic O(log n) ===
x   n   Result      Steps ≈ log2(n)
2   5   32      2
2   10  1024        3
2   20  1048576     4
2   100 0       6
2   1000    0       9

[Done] exited with code=0 in 1.224 seconds
```

Analysis and Comparison:

Metric	rpower	brpower
Complexity	O(n) — Linea	O(log n) — Logarithmic
How it works	Subtract 1 each time	Divide n in half each time

Groqth	N doubles = steps double	n doubles = steps +1 only
N = 1,000,000	1,000,000 steps	20 steps
Sim[licity	Very simple	Slightly more logic
Large n	Risk of stack is overflow	Efficient and safe

Conclusion:
Both algorithms produce the same correct result, but they perform very differently. rpower is simple and easy to understand it just counts down one by one, so its runtime grows directly with the exponent. brpower is much smarter: it repeatedly halves the problem size, so it finishes in logarithmic time. For small exponents, both work fine. But for large exponents like n = 1,000,000, brpower is practically instant while rpower would take far too long to be useful. This shows that choosing the right algorithm approach makes an enormous difference in performance.

COMPUTER ENGINEERING

QUEZON CITY

Supplementary Prieto:

Problem 1

Theoretical analysis:

The algorithm uses two nested for loops. The inner loop compares the current element with the remaining elements; the outer loop iterates through the array.

The number of comparisons is: $n(n-1)/2 = \frac{1}{2}n^2 - \frac{1}{2}n$

O(n) is the worst-case time complexity. Because the number of comparisons increases quadratically, the algorithm slows as the input size increases.

Experimental Analysis:

PrietoSevilla.cpp > Unique(const vector<int>&)

```cpp
#include <iostream>
#include <vector>
#include <chrono>

using namespace std;

bool Unique(const vector<int>& A)
{
    int n=A.size();

    for (int i = 0; i <n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (A[i] == A[j])
            {
                return false;
            }
        }
    }

    return true;
}

int main()
{
    int sizes[] = {100, 500, 1000, 2000};

    for (int n : sizes)
    {
        vector<int> A(n);

        for (int i = 0; i < n; i++)
        {
            A[i] = i;
        }

        auto start = chrono::high_resolution_clock::now();

        bool result = Unique(A);

        auto end = chrono::high_resolution_clock::now();

        auto duration = chrono::duration_cast<chrono::microseconds>(end - start);

        cout << "Input Size: " << n << endl;
        cout << "Result: " << (result ? "Unique" : "Not Unique") << endl;
        cout << "Execution Time: " << duration.count()
             << " microseconds" << endl;
    }

    return 0;
}
```

```
Input Size: 100
Result: Unique
Execution Time: 40 microseconds
Input Size: 500
Result: Unique
Execution Time: 665 microseconds
Input Size: 1000
Result: Unique
Execution Time: 2786 microseconds
Input Size: 2000
Result: Unique
Execution Time: 11405 microseconds
PS C:\Users\gloria escoto\New folder (5)\Hoa11p1>
```

Input Size	Comparison
10	45
20	190
30	435
100	4,950

Observations: The best case can be O(1) if a duplicate is found immediately, but when all elements are unique, the algorithm must check every pair, resulting in $O(n^2)$.

Input Size	Time	Result
100	40	Unique
500	665	Unique
1000	2786	Unique
2000	11405	Unique

Analysis and comparison:
The experimental results indicate that the execution time increases with the increase of input size. This corresponds to the theoretical O(n2) complexity of the algorithm. As the size of the input increases from 100 to 2,000 the number of element comparisons increase rapidly. Thus, the experimental results confirm the theoretical analysis that the efficiency of the algorithm deteriorates for larger arrays.

Problem 2:

Theoretical Analysis

A. rpower Each call reduces n by exactly 1

Time Complexity: O(n)

B. brpower

Each call halves the value of n

Time Complexity: O(log n)

Algorithm	Complexity Class	Steps at n=2000	n doubles
rpower	O(n) linear	~2000	Steps ×2
brpower	O(log n) logarithmic	~11	Steps ×1

Experimental analysis:

```cpp
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

// --- Linear exponentiation: O(n) ---
long long rpower(long long x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

// --- Fast exponentiation: O(log n) ---
long long brpower(long long x, int n) {
    if (n == 0) return 1;
    if (n % 2 == 1) { // odd
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else { // even
        long long y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    cout << "PROBLEM 2: Power Algorithms" << "\n\n";
    cout << "n       | rpower O(n) | brpower O(log n)\n";
    for (int n : {100, 500, 1000, 2000}) {
        auto t1 = high_resolution_clock::now();
        volatile long long a = rpower(2, n);
        auto t2 = high_resolution_clock::now();

        auto t3 = high_resolution_clock::now();
        volatile long long b = brpower(2, n);
        auto t4 = high_resolution_clock::now();

        auto us_r = duration_cast<microseconds>(t2 - t1).count();
        auto us_br = duration_cast<microseconds>(t4 - t3).count();

        printf("%6d | %16lld | %20lld\n", n, us_r, us_br);
    }
    return 0;
}
```

```
n       | rpower O(n) | brpower O(log n)
   100 |               1 |                    0
   500 |              31 |                    0
  1000 |              33 |                    0
  2000 |              56 |                    0
```

n	rpowerO(n)	brpower O(log n)
100	1	0
500	31	0
1000	33	0
2000	56	0

Observations:
rpower: n ×2 runtime ×2
brpower: runtime stays nearly constant
n = 2000, rpower is ~90× slower than brpower

Analysis and Comparison:

Comparison	Theoretical
rpower growth	O(n) — linear, doubles with n
brpower growth	O(log n) — near-constant
Relative speed	brpower vastly faster

Observations:
rpower: Shows a steady, proportional increase. When n gets large, this becomes impractical -- n=1,000,000 would take ~1 second vs ~20 steps for brpower.
brpower: Grows at such a slow rate that the runtime appears constant over large ranges The logarithmic curve is almost flat.

Conclusion:
This helps me how to use both theoretical and experimental analysis to assess an algorithm's efficiency. I discovered that the size of the input has an impact on the performance of various algorithms, which can have varying time complexities. I was able to comprehend that $O(n^2)$ is slower than O(n) and O(log n). My comprehension of algorithm performance and time complexity still needs to be strengthened.